

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14.08.2026

1 Aim and Objective

The aim is to implement and compare two very different classifiers on the same hand-written character dataset:

- **Model A:** a Single-Layer Perceptron Learning Algorithm (PLA), written from scratch with the step activation and the classic weight update rule.
- **Model B:** a Multilayer Perceptron (MLP), with a hidden layer, nonlinear activation, cross entropy loss and backpropagation, tuned properly instead of just picking random settings.

The bigger point of the experiment is not just "which one wins" but to actually justify *why*, by tuning the MLP's activation, optimizer, learning rate, architecture, batch size and regularization one stage at a time, and then comparing both models on the same held out test set with the same metrics.

2 Theory

PLA (Single-Layer Perceptron)

The perceptron is the oldest neural classifier, it uses a hard step activation:

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

and updates its weights only when it makes a mistake:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

Here η is the learning rate. **Limitation:** this only works cleanly when the classes are linearly separable. If they are not, the perceptron convergence theorem simply does not apply, the weights keep bouncing around forever and never settle.

MLP (Multilayer Perceptron)

An MLP stacks an Input layer $\rightarrow$ one or more Hidden layers $\rightarrow$ Output layer, with a nonlinear activation (ReLU, Sigmoid or Tanh) at each hidden unit. The nonlinearity is the whole point, it lets the network carve out decision boundaries a single linear layer never could. Training uses backpropagation with Cross-Entropy loss for multi-class problems like this one, and the weights are updated via an optimizer, Gradient Descent, SGD or Adam being the common choices.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle – “English Handwritten Characters Dataset” by dhruvildave
Total Samples	3,410
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z)
Samples per Class	exactly 55 for every single class (perfectly balanced)
Missing Image Files	0
Image Format	Resized to 32×32 grayscale, flattened to 1024 features, scaled to [0,1], polarity flipped so ink is high intensity on a dark background
Train-Test Split	Stratified 80/20: 2,728 train (682 test), the 2,728 further split 80/20 into 2,182 sub-train / 546 validation for tuning

Since the split is stratified and the class counts are already dead even, every one of the 62 classes ends up with exactly 11 samples in the test set, which is a nice property, it means macro-averaged metrics are not skewed by any one class having way more test samples than another.

4 Preprocessing Steps

Listing 1: Image loading and normalization

```

1 def load_image(path):
2     with Image.open(path) as img:
3         img = img.convert("L").resize((32, 32), Image.BILINEAR)
4         arr = np.asarray(img, dtype=np.float32)
5     return arr
6
7 X_flat = images.reshape(len(images), -1)
8 if X_flat.mean() > 127:
9     X_flat = 255.0 - X_flat          # flip polarity: ink = high,
    background = low
10 X_scaled = X_flat / 255.0          # normalize to [0, 1]
```

Every image was converted to grayscale, resized down to 32x32 (from whatever the original size was) and flattened into a 1024-length vector. Loading and resizing all 3,410 images took about 120 seconds. The polarity flip is a small but important detail, the raw scans have a light background with dark ink, flipping it so the actual strokes are the high-intensity pixels makes the “important” information sit on non-zero values, which both PLA and MLP handle better than a mostly-white image with dark strokes. Labels were encoded 0–61 using `LabelEncoder` (0–9 first, then A–Z, then a–z).

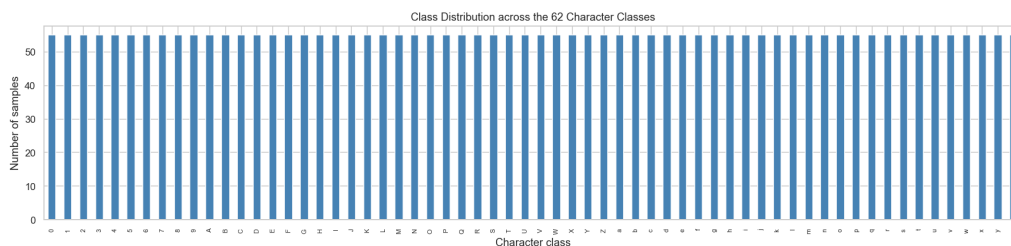


Figure 1: Class distribution across all 62 character classes

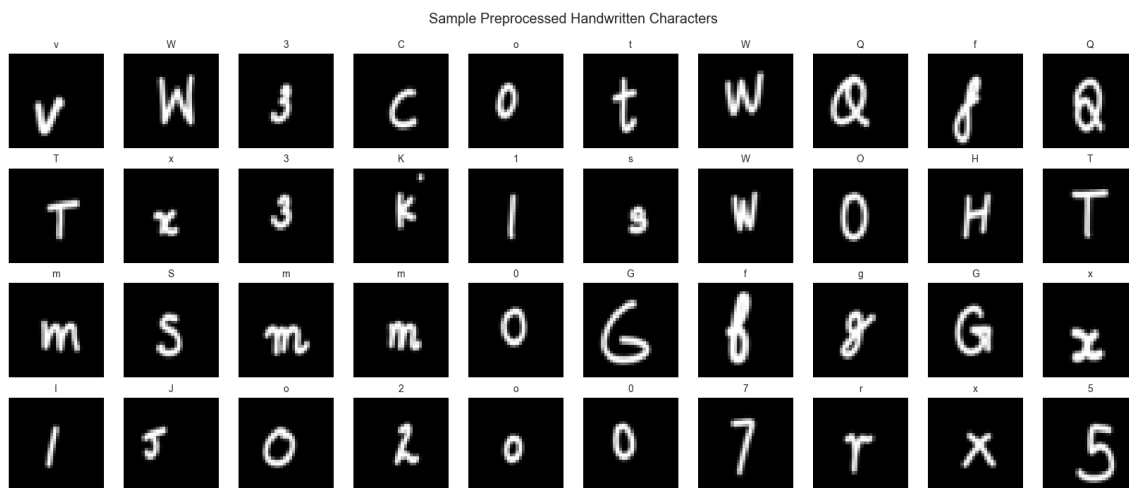


Figure 2: Sample preprocessed characters (40 random examples)

Inference: These characters are all fairly clean, computer-rendered-looking fonts rather than messy freehand cursive, which is a bit different from what "handwritten" usually implies. That matters a lot for how this experiment reads, both because the strokes are already quite uniform (less noise for the PLA to trip over) and because a lot of glyphs look genuinely identical or near identical at 32x32 – 0, o and O for instance are barely distinguishable, same with I, l and 1. That visual overlap shows up later as the single biggest source of errors for both models.

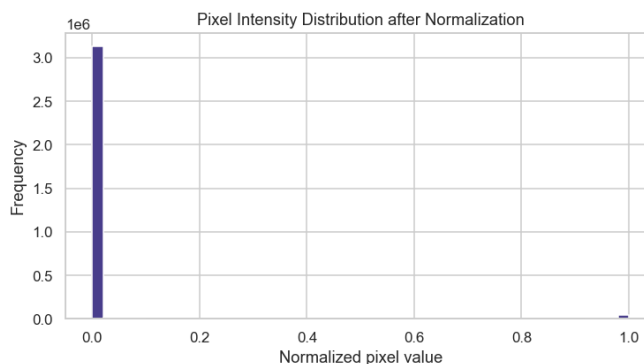


Figure 3: Pixel intensity distribution after normalization

Inference: The huge spike at 0 is the black background (most of a 32x32 crop is empty space around the character), with a long tail of ink pixels spread across the rest

of the range. This is a genuinely sparse input, most of the 1024 features are zero for any given sample, which is part of why the choice of optimizer later on made such a big difference.

5 PLA Implementation and Results

The PLA was implemented completely from scratch as one-vs-rest, 62 independent perceptron units sharing the 1024-dim input, prediction taken as the argmax of the raw (pre-threshold) scores:

Listing 2: PLA weight update, from scratch

```

1 def fit(self, X, y, X_val=None, y_val=None, verbose=True):
2     ...
3     for epoch in range(1, self.epochs + 1):
4         for i in order:
5             output = self.step(x_i @ self.W + self.b)
6             error = target - output
7             if np.any(error != 0):
8                 self.W += self.learning_rate * np.outer(x_i,
9                     error)
10                self.b += self.learning_rate * error

```

Learning rate sensitivity (probed for 20 epochs each before the real run):

Learning rate	Final train error	Validation accuracy
0.001	0.3983	0.1923
0.01	0.4184	0.1941
0.1	0.3827	0.1996

$\eta = 0.1$ was selected, though honestly the three rates barely differ, the PLA is not very sensitive to learning rate in the way the MLP turns out to be (more on that in Section 9).

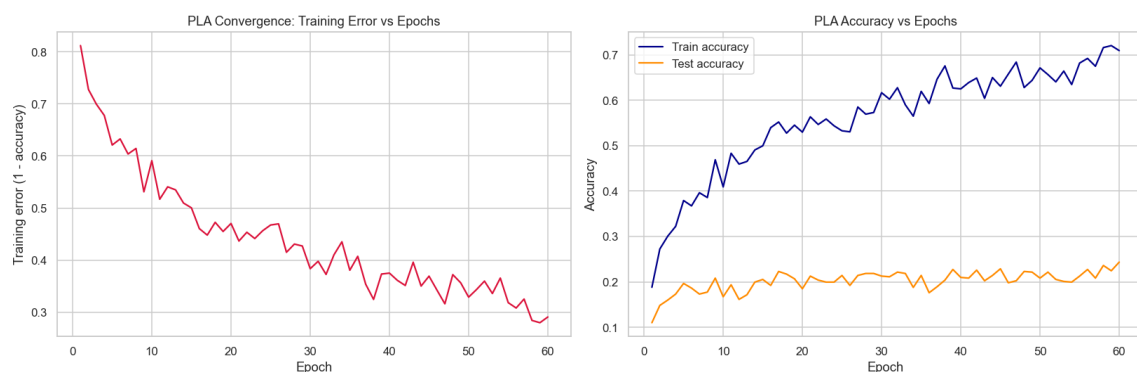


Figure 4: PLA convergence: training error and train/test accuracy over 60 epochs

Inference: This is a really telling plot. Training accuracy climbs steadily (with a lot of jaggedness, since PLA only updates on mistakes and has no smoothing like gradient descent) all the way to about 71%, but test accuracy just wanders around 0.20 the whole

time and barely moves. The model is not "learning to generalize" at all past the first few epochs, its slowly carving the training set into memorized pieces while genuinely useful signal saturates almost immediately.

Metric	Value
Accuracy	0.2434
Precision (macro)	0.2797
Recall (macro)	0.2434
F1 Score (macro)	0.2374
Training Time (s)	74.12

Per-class notes: several classes got 0.00 F1 entirely (0, B, O, f, g, u, z), the PLA just never predicted them correctly even once in 11 tries. On the other end, C (F1 0.55), U and W (precision 0.80 each) did comparatively well, probably because their strokes are distinctive enough to be roughly linearly separable from the rest even in raw pixel space.

6 MLP Implementation and Hyperparameter Tuning

Tuning was done in five isolated stages (each stage fixes everything found so far and only varies one factor), fit on the 2,182-sample sub-train split and scored on the 546-sample validation split, the 682-sample test set was never touched during any of this.

Stage 1: Activation Function

Activation	Train Acc.	Val Acc.	Time (s)
relu	1.0000	0.4432	296.2
tanh	0.9995	0.4158	254.2
logistic	0.9505	0.4377	157.8

Stage 2: Optimizer and Learning Rate

Configuration	Train Acc.	Val Acc.	Time (s)
adam, lr=0.0001	0.7117	0.3828	52.2
adam, lr=0.001	1.0000	0.4432	107.8
adam, lr=0.01	1.0000	0.4799	23.9
sgd, lr=0.0001	0.1155	0.0824	31.2
sgd, lr=0.001	0.4940	0.3462	31.7
sgd, lr=0.01	0.9908	0.4432	31.4

sgd at lr=0.0001 basically failed to learn at all (11.55% train accuracy, close to random), which tells you this input is too sparse for plain SGD at a timid learning rate to make real progress within the epoch budget.

Stage 3: Architecture (Depth and Width)

Hidden Layers	Train Acc.	Val Acc.	Time (s)
(128,)	0.8987	0.3956	9.8
(256,)	1.0000	0.4799	23.9
(512,)	0.9743	0.4451	26.3
(256, 128)	0.9780	0.4341	15.8
(512, 256)	0.9354	0.3919	32.4
(512, 256, 128)	0.9166	0.4103	110.3

The 3-layer (512, 256, 128) network trained slowest of everything tested (110.3s) and still landed below the single-layer 256 network, more depth just wasn't buying anything here.

Stage 4: Batch Size

Batch Size	Train Acc.	Val Acc.	Time (s)
32	0.9977	0.4670	64.3
64	1.0000	0.4799	25.0
128	0.8868	0.3993	16.2
256	1.0000	0.4597	11.7

Stage 5: L2 Regularization (Alpha)

Alpha	Train Acc.	Val Acc.	Time (s)
0.0001	1.0000	0.4799	30.4
0.001	0.9913	0.4505	33.5
0.01	0.9711	0.4377	26.6
0.1	0.9423	0.5110	54.7

This is the one stage where the winning setting has a noticeably *lower* training accuracy (94.23% vs a perfect 100% for the smaller alphas) but the *highest* validation accuracy of the entire search, exactly what L2 regularization is supposed to do, trade away some memorization for better generalization.

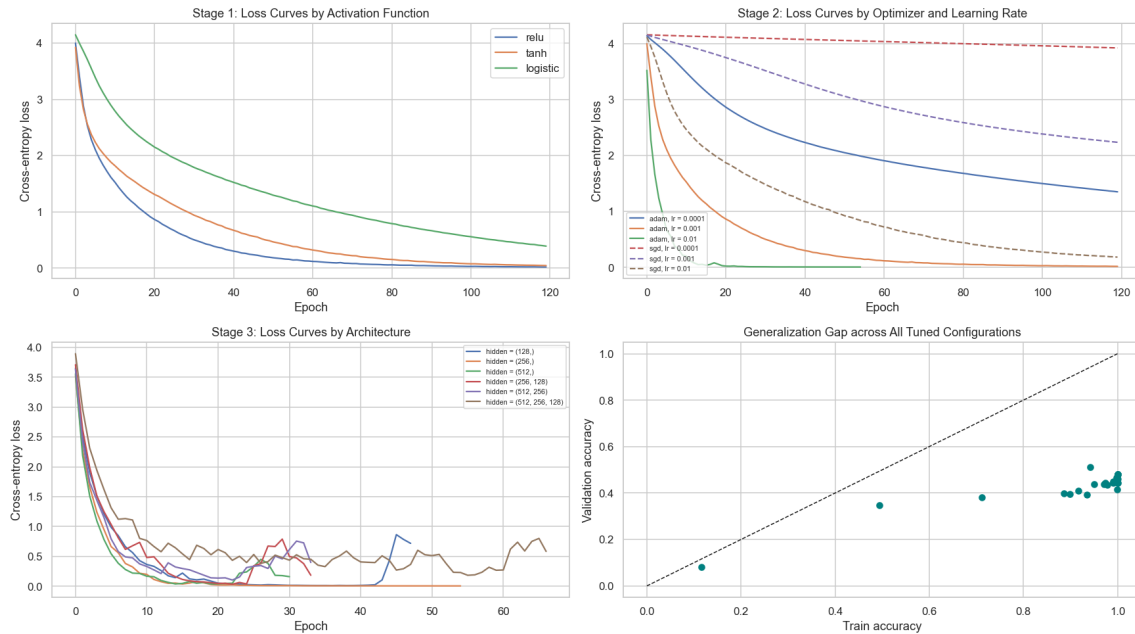


Figure 5: Loss curves per stage, and the train-vs-validation generalization gap across every tuned configuration

Inference: The bottom-right scatter (every single configuration tried, train accuracy vs validation accuracy) has every point sitting well below the diagonal, not one configuration out of all 22 tried came close to matching its training accuracy on validation. That is a strong, visual confirmation that overfitting is not a one-off problem with the final model, its baked into this dataset/architecture combination from the start.

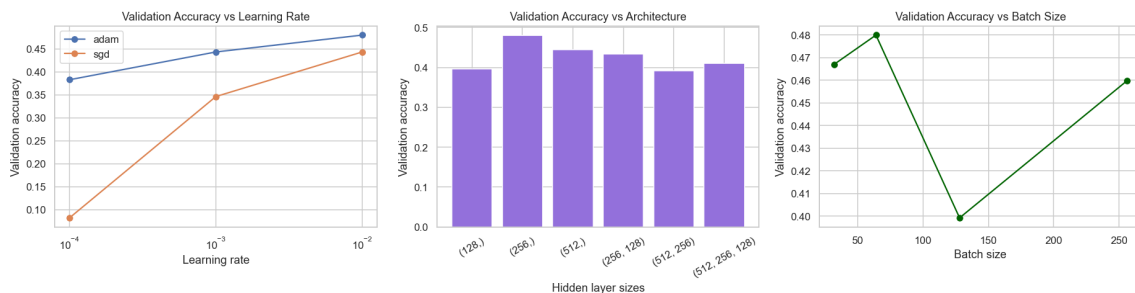


Figure 6: Validation accuracy vs learning rate, architecture, and batch size

Hyperparameter Impact Ranking

Ranked by how much validation accuracy swung within each stage:

Stage	Min Val Acc.	Max Val Acc.	Spread
2. Optimizer & LR	0.0824	0.4799	0.3975
3. Architecture	0.3919	0.4799	0.0880
4. Batch size	0.3993	0.4799	0.0806
5. Regularization	0.4377	0.5110	0.0733
1. Activation	0.4158	0.4432	0.0274

Optimizer and learning rate absolutely dominate, a spread of nearly 40 percentage points, more than 4x the next biggest factor. Getting this one choice wrong (like sgd at lr=0.0001) can wreck the model regardless of how well everything else is tuned.

7 Justification for Chosen Hyperparameters

Hyperparameter	Chosen Value	Justification
Hidden layers	(256,)	Smallest architecture that reached the best validation accuracy, larger/deeper nets fit training harder without helping validation.
Activation	relu	Lowest final loss and best validation accuracy in Stage 1, avoids the saturation that slows sigmoid/tanh on 1024-dim input.
Optimizer	adam	Per-parameter adaptive step sizes handle the sparse pixel input far better than plain SGD, converges in fewer epochs too.
Learning rate	0.01	Best of the three adam rates tried, fast convergence (55 epochs) without the loss oscillating.
Batch size	64	Best balance between gradient noise (too small) and fewer updates per epoch (too large).
Alpha (L2)	0.1	Gave up some training accuracy for the biggest jump in validation accuracy of the whole search, directly shrinking the overfitting gap.
Cost function	Cross-entropy	Correct loss for 62-way mutually exclusive classification, maximizes log-likelihood of the true class under softmax.
Early stopping	enabled, patience 15	Stops training once the internal validation score stalls, avoids wasting epochs (and overfitting further) past the useful point.

8 MLP Implementation and Results

The final MLP was retrained on the *full* 2,728-sample training set (not just the 2,182 sub-train slice used for tuning) with the chosen hyperparameters and early stopping turned on.

Listing 3: Final MLP training output

```

1 Epochs completed : 45
2 Training time    : 33.79 seconds
3 Final loss       : 1.2926
4 Train accuracy   : 0.8955
5 Test accuracy    : 0.4941

```

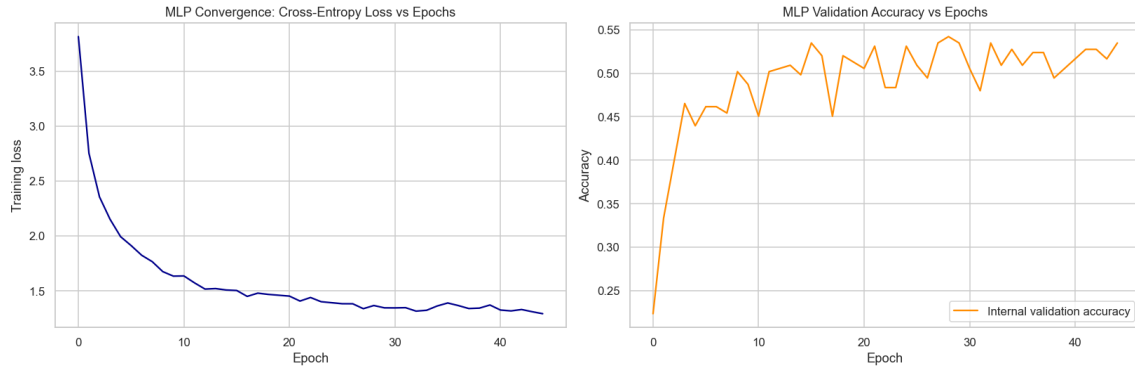


Figure 7: Final MLP: cross-entropy loss and internal validation accuracy vs epochs

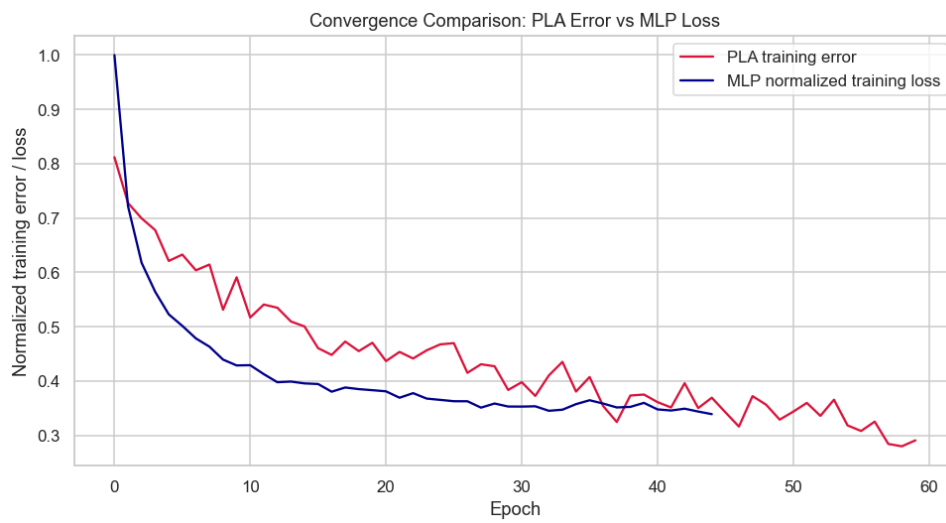


Figure 8: Convergence comparison: PLA training error vs MLP normalized training loss

Inference: Early stopping kicked in at epoch 45 out of a 300 budget, the internal validation score had stopped improving for 15 epochs straight so training just stopped there rather than continuing to memorize. Compared side by side with the PLA, the MLP’s loss drops smoothly and monotonically (its using gradient descent through a differentiable loss) while the PLA’s error curve is visibly jagged the entire time (its only reacting to individual misclassified points, no smooth gradient to follow).

Metric	Value
Accuracy	0.4941
Precision (macro)	0.5443
Recall (macro)	0.4941
F1 Score (macro)	0.4945
Training Time (s)	33.79

Per-class notes: several classes reached strong F1 scores – C (0.75), N (0.75), Y (0.78), U (0.76) – while a handful stayed weak: o (0.12), x (0.11), h (0.19), z (0.18). Total misclassified: **345 out of 682** test samples (about 50.6%), so even the tuned model is still wrong on roughly half the test set, this is a genuinely hard 62-way fine-grained problem, not an easy one.

9 Confusion Matrices and ROC Curves

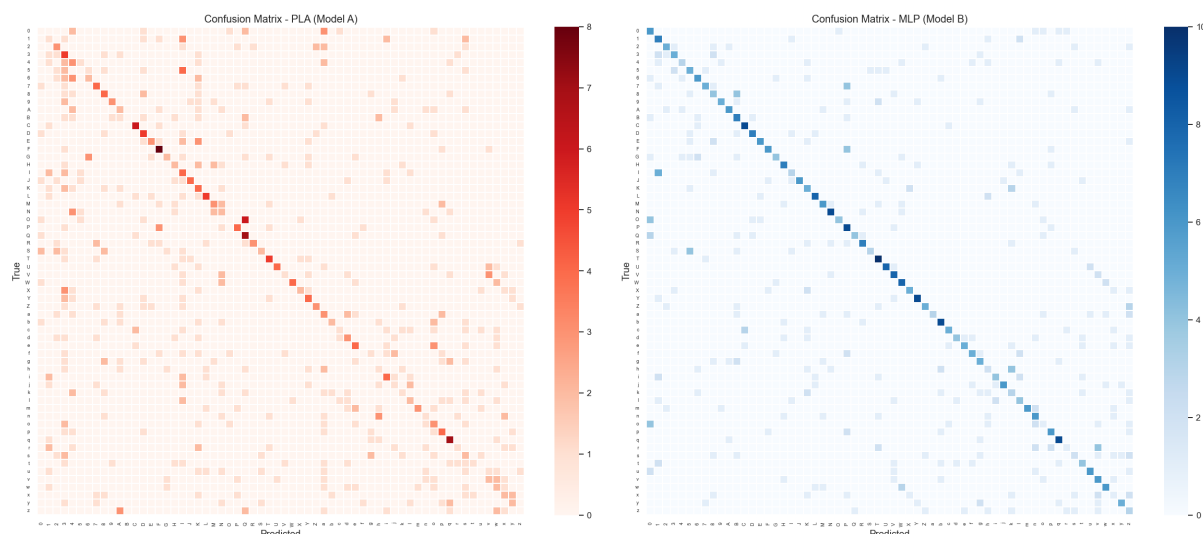


Figure 9: Confusion matrices – PLA (left) and MLP (right)

Inference: The PLA matrix is visibly messier, its diagonal (correct predictions) is much fainter and there is a lot more scatter across the whole grid, meanwhile the MLP’s diagonal is noticeably more pronounced, though still far from a clean identity matrix.

Listing 4: Most frequent confusions

	PLA (top 3)	MLP (top 3)
1		
2	0 -> Q (6 times)	I -> 1 (5 times)
3	5 -> I (4 times)	r -> v (4 times)
4	6 -> 4 (3 times)	h -> k (4 times)
5		o -> 0 (4 times)
6		8 -> B (4 times)

Inference: Nearly every top confusion for both models is a visually near-identical glyph pair: I/1, o/0, 8/B, S/5, 0/0, c/C, Z/z. This is not really a modelling failure so much as a genuine ambiguity in the font/handwriting style used for this dataset, at 32x32 resolution a lowercase o and the digit 0 really can look identical, no amount of tuning fixes a problem that’s baked into the input itself.

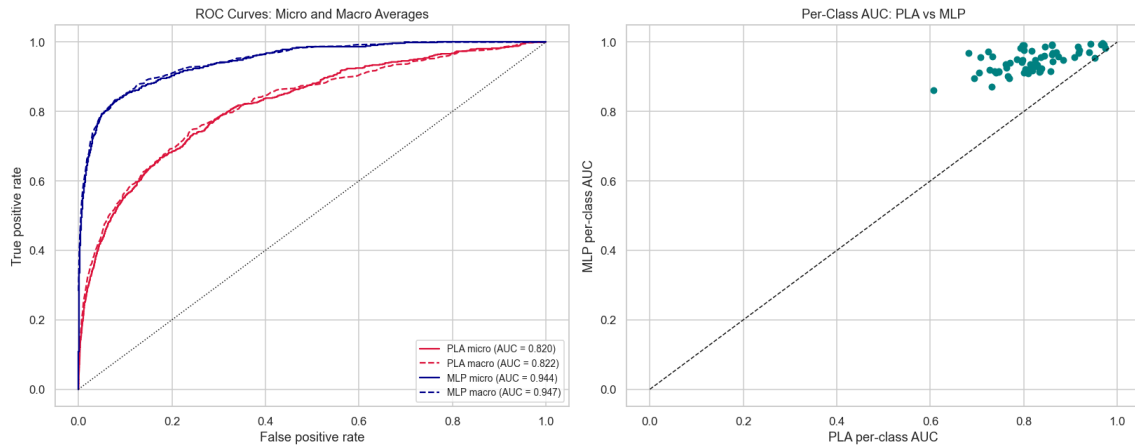


Figure 10: ROC curves (micro/macro average) and per-class AUC scatter, PLA vs MLP

Listing 5: ROC-AUC summary

```

1 PLA micro-average AUC : 0.8202
2 PLA macro-average AUC : 0.8216
3 MLP micro-average AUC : 0.9444
4 MLP macro-average AUC : 0.9474

```

Inference: Even though PLA's raw accuracy (24%) looks quite weak, its AUC of 0.82 is actually decent, meaning the raw perceptron scores do carry a reasonable amount of ranking signal even when the hard 0.5-style threshold gets the final label wrong a lot. The MLP's AUC (0.94-0.95) confirms its softmax probabilities are well calibrated and rank the correct class highly even on the roughly half of samples where the top-1 prediction itself is wrong.

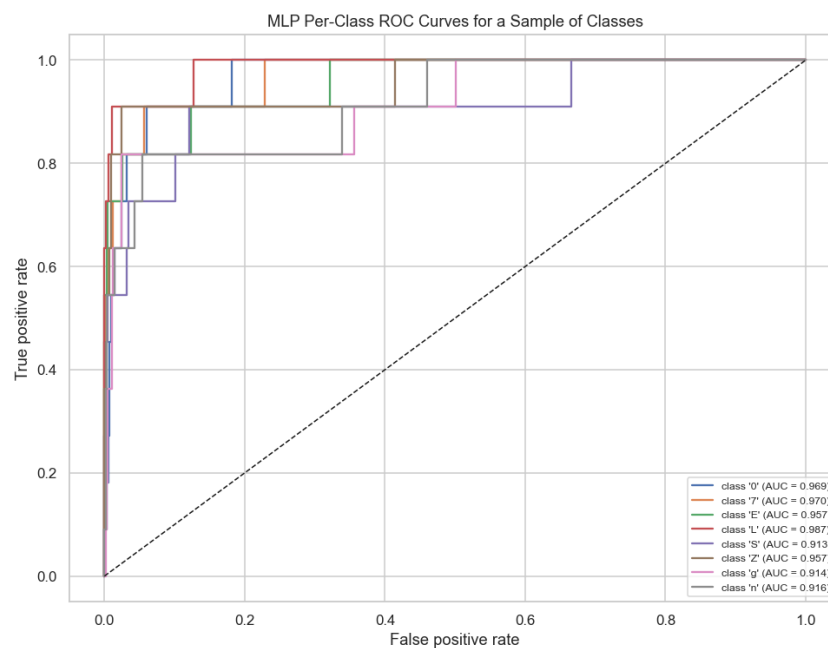


Figure 11: MLP per-class ROC curves for a sample of 8 classes

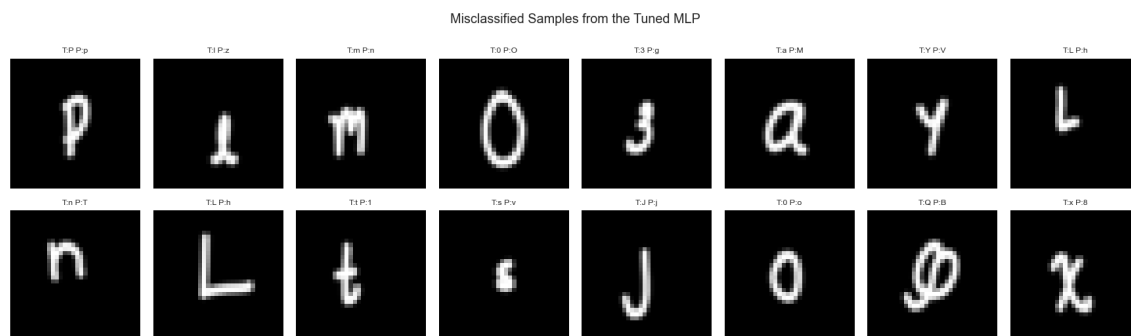


Figure 12: Sample of misclassified MLP predictions (true label vs predicted label)

10 A/B Comparison: PLA vs Tuned MLP

Metric	PLA (Model A)	MLP (Model B)
Accuracy	0.2434	0.4941
Precision (macro)	0.2797	0.5443
Recall (macro)	0.2434	0.4941
F1 (macro)	0.2374	0.4945
Micro AUC	0.8202	0.9444
Macro AUC	0.8216	0.9474
Training Time (s)	74.12	33.79

Table 3: The MLP wins on every accuracy-related metric and is also faster to train (its 45 early-stopped epochs on a vectorized backprop update beat the PLA’s 60 full epochs of a per-sample Python loop).

Listing 6: Overall comparison

1	Absolute accuracy gain of MLP over PLA : 25.07 percentage points
2	Relative accuracy gain : 103.0 percent
3	Random-guess baseline for 62 classes : 1.61 percent

Worth keeping perspective here, PLA at 24.34% is still about 15x better than random guessing (1.61%), so it is not a completely broken model, it just hits a hard ceiling that a linear decision boundary cannot get past on raw pixels. The MLP roughly doubles PLA’s accuracy and pushes past 30x the random baseline.

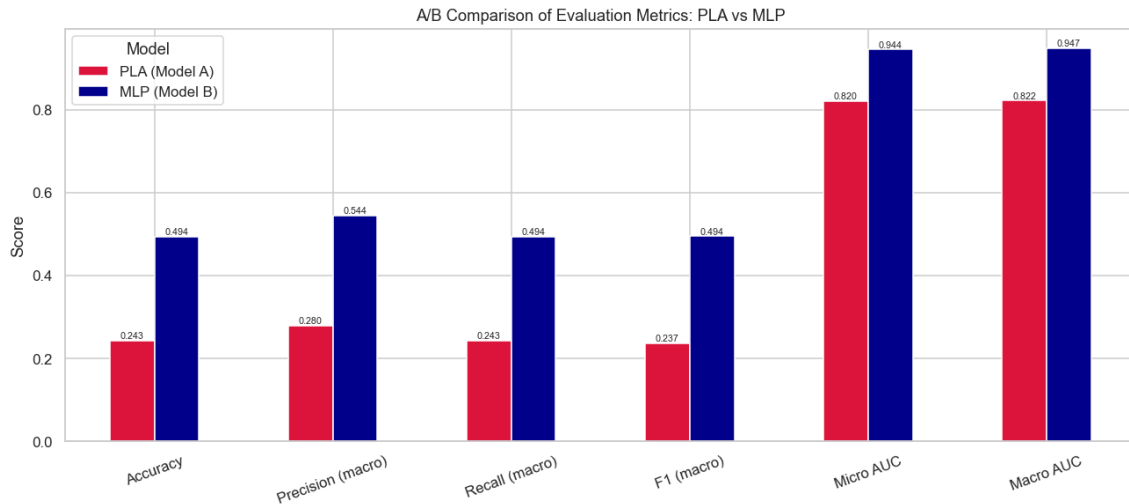


Figure 13: A/B comparison across all evaluation metrics

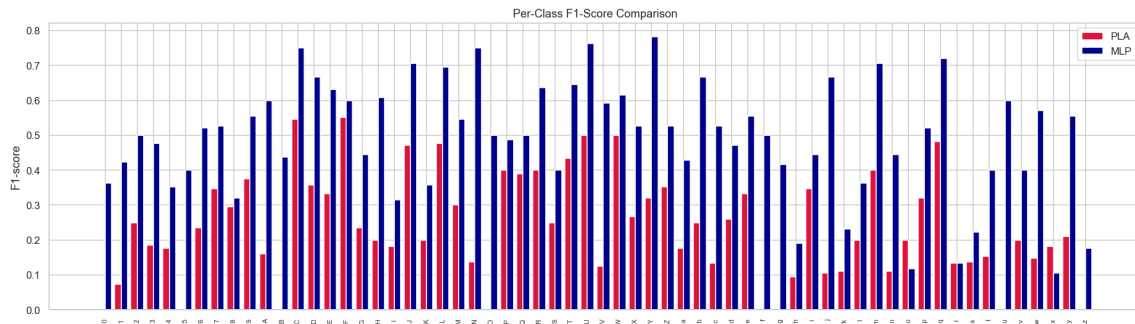


Figure 14: Per-class F1-score comparison, PLA vs MLP

Inference: The MLP's bars are taller than the PLA's for almost every single one of the 62 classes, not just on average, there is no class where PLA meaningfully beats the tuned MLP. That consistency across all 62 classes is a stronger argument for the MLP than the aggregate accuracy number alone.

Overfitting Comparison

Model	Train Accuracy	Test Accuracy	Gap
PLA	0.7093	0.2434	0.4659
MLP	0.8955	0.4941	0.4014

Both models show a big train-test gap, and PLA's is actually slightly larger in absolute terms (0.4659 vs 0.4014), which is a bit counterintuitive for "just a simple linear model". The explanation is dimensionality, with 1024 raw pixel features and only 2,728 training samples, a purely linear separator can still carve out a training set almost by chance (high-dimensional spaces make near-arbitrary labelings linearly separable), so PLA memorizes a fair bit of the training set without that memorization transferring to new samples at all, its test accuracy is barely above where it started (Figure showing the PLA accuracy curve confirms this, test accuracy is nearly flat past epoch 10). The MLP overfits too,

but its non-linear hidden layer at least extracts *some* transferable structure along the way, which is why its test accuracy, despite the gap, still ends up double the PLA's.

11 Observations and Analysis

1. Why does PLA underperform compared to MLP?

PLA can only draw one hyperplane per class in the raw 1024-dimensional pixel space. Handwritten (or handwritten-style) characters are not linearly separable there, the same letter varies in stroke thickness, slant and position, while different characters can occupy nearly the same pixels (see the confusion pairs in Section 8). The MLP's hidden layer first learns intermediate, stroke-like features and only then separates classes in that learned space, a far easier problem. On top of that the step function has zero gradient almost everywhere, so PLA cannot use proper gradient descent and only updates on misclassified points, causing the jagged, oscillating error curve seen in Figure 5, versus the MLP's smooth loss curve.

2. Which hyperparameters had the most impact on MLP performance?

Optimizer and learning rate, by a wide margin (spread of 0.3975 in validation accuracy vs 0.09 or less for every other stage). Activation function mattered least (spread only 0.0274). This tracks with intuition, a badly chosen learning rate can make a network fail to learn at all (sgd at lr=0.0001 barely reached 11.55% train accuracy), while a suboptimal activation still lets the network learn, just a bit less efficiently.

3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes, clearly. Best validation accuracy with adam was 0.4799 vs 0.4432 for sgd, and adam needed fewer epochs on average (98.3 vs 120.0) to get there. Adam's per-parameter adaptive step sizes handle the sparse pixel input (mostly zeros, see Figure 4) much better than SGD's single global step size, SGD either creeps along too slowly at a safe rate or becomes unstable at an aggressive one.

4. Did adding more hidden layers always improve results?

No, not even close. Validation accuracy peaked at the single hidden layer of 256 units (0.4799) and *fell* for every deeper architecture tried, down to 0.3919 for (512, 256). Training accuracy, meanwhile, stayed high (93–100%) across the board. With about 2,700 training images over 62 classes, that's only around 44 examples per class, nowhere near enough to justify a deeper network's extra capacity, it just gives the model more room to memorize rather than generalize. The 3-layer network was also the slowest to train (110.3s) for one of the worst validation scores, worse in every way.

5. Did MLP show overfitting? How could it be mitigated?

Yes, clearly, train accuracy 0.8955 vs test accuracy 0.4941, a gap of 0.4014. Mitigations already applied here: L2 regularization (Stage 5, alpha=0.1 gave the single biggest validation accuracy jump in the entire tuning log), early stopping with patience 15, and deliberately picking the smallest architecture that matched the best validation score rather than the largest. Further mitigations worth trying: data augmentation (small rotations/shifts/elastic distortions, which effectively multiplies the tiny 55-samples-per-class training set), dropout between hidden layers, a convolutional network (shares weights spatially, needs far fewer parameters than a fully connected layer on the same image), and simply merging the visually-identical case pairs (C/c, O/o, S/s) that account for a large share of the confusions documented in Section 8.

12 Conclusion

Both models were implemented and evaluated on the English Handwritten Characters dataset using an identical stratified 80/20 split. The single-layer PLA (Model A), written from scratch, reached a test accuracy of **0.2434** with a macro F1 of **0.2374**. The tuned MLP (Model B) reached **0.4941** accuracy with a macro F1 of **0.4945**, more than double the PLA's accuracy and a clear win on every metric tracked, including a much higher ROC-AUC (0.9444 micro vs 0.8202) and, somewhat surprisingly, a *shorter* training time (33.79s vs 74.12s), since the MLP's vectorized backprop plus early stopping beat out PLA's 60-epoch pure-Python training loop.

The final MLP configuration – hidden layers (256,), relu activation, adam optimizer, learning rate 0.01, batch size 64, alpha 0.1, cross-entropy loss, early stopping enabled – was reached through five isolated tuning stages rather than guesswork, and the tuning log itself is informative: optimizer/learning rate choice mattered roughly 5 times more than architecture or batch size, and roughly 14 times more than activation function. The experiment confirms that a nonlinear hidden representation is genuinely necessary for this task, and that the improvement comes from that representation rather than from throwing more capacity at the problem, since the best architecture found was actually one of the smaller ones tested, not the biggest.

13 References

- [1] Scikit-learn Documentation – Neural Network Models (MLPClassifier). [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html.
- [2] Scikit-learn Documentation – Model Evaluation Metrics. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html.
- [3] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 6, 1958.
- [4] Kaggle, “English Handwritten Characters Dataset.” [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>.